FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	REYES, SOFIA BEATRICE ALEXA R.
Section:	AN42
Professor:	CRISOLA TAN
Date:	OCTOBER 29, 2026
GITHUB LINK:	https://github.com/brryes/Reyes-SofiaBeatriceAlexa-CS0065-AN42.git

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

—----------

1. Which system (RBR or CBR) was easier to implement? Why?

RBR was easier to implement because it uses simple IF–THEN rules to make decisions based on given facts.

2. Which is more flexible in solving unstructured problems?

CBR is more flexible because it uses similar past cases to find possible solutions to new problems.

3. Which is better for situations with continuously changing data?

CBR is more suitable because new cases and solutions can be added to its case base over time.

4. Examples of real-world applications for both.

RBR can be used in automated systems and medical diagnosis, while CBR can be applied in troubleshooting systems, disaster response planning, and recommendation systems.

1.

Your Code - Laboratory Tasks ⇩

```
"""
TA4 - Knowledge Representation, Rule-Based Reasoning, and Case-Based Reasoning
Use case: Disaster Response Decision Support System

This educational prototype uses sample data only. It does not replace official
emergency services or real-time disaster information.
"""

from copy import deepcopy

# PART 1: KNOWLEDGE REPRESENTATION (KR)
# Facts are grouped into three categories: hazard, impact, and resources.
incident_facts = {
    "hazard": {
        "heavy_rain": True,
        "flood_level_cm": 65,
        "road_blocked": True,
    },
    "impact": {
        "people_needing_evacuation": 120,
        "injured_people": 4,
    },
    "resources": {
        "shelter_available": True,
        "rescue_vehicles_available": 2,
    },
}


# PART 2: RULE-BASED REASONING (RBR)
```

```
# Each function checks the knowledge base and returns an action if its rule fires.
def rule_flood_warning(facts):
    if facts["hazard"]["heavy_rain"] and facts["hazard"]["flood_level_cm"] >= 50:
        return "Issue a flood warning for the affected area."
    return None


def rule_road_closure(facts):
    if facts["hazard"]["road_blocked"]:
        return "Notify responders and mark the blocked road as inaccessible."
    return None


def rule_evacuation(facts):
                if     facts["impact"]["people_needing_evacuation"]     >=     50     and
facts["resources"]["shelter_available"]:
        return "Coordinate evacuation and direct residents to the available shelter."
    return None


def rule_medical_support(facts):
    if facts["impact"]["injured_people"] > 0:
        return "Dispatch medical support for injured people."
    return None


def run_rbr(facts):
    """Inference engine: check every rule and collect all inferred actions."""
            rules    =    [rule_flood_warning,    rule_road_closure,    rule_evacuation,
rule_medical_support]
    actions = []
    for rule in rules:
        action = rule(facts)
        if action:
            actions.append(action)
    return actions


# PART 3: CASE-BASED REASONING (CBR)
```

```
# Each case stores problem features and a solution from a previous incident.
case_base = [
    {
        "name": "Case 1 - Severe Flooding",
        "problem": {"heavy_rain": True, "flood_level_high": True, "road_blocked": True,
                    "many_evacuees": True, "injuries": True},
          "solution": "Issue a flood warning, close blocked roads, evacuate residents,
and send medical support."
    },
    {
        "name": "Case 2 - Localized Road Flooding",
           "problem": {"heavy_rain": True, "flood_level_high": False, "road_blocked":
True,
                    "many_evacuees": False, "injuries": False},
          "solution": "Warn nearby residents and reroute traffic away from the flooded
road."
    },
    {
        "name": "Case 3 - Earthquake with Injuries",
          "problem": {"heavy_rain": False, "flood_level_high": False, "road_blocked":
True,
                    "many_evacuees": True, "injuries": True},
          "solution": "Secure damaged roads, evacuate affected residents, and dispatch
medical teams."
    },
]


def get_problem_features(facts):
    """Convert current facts to the same feature format used by stored cases."""
    return {
        "heavy_rain": facts["hazard"]["heavy_rain"],
        "flood_level_high": facts["hazard"]["flood_level_cm"] >= 50,
        "road_blocked": facts["hazard"]["road_blocked"],
        "many_evacuees": facts["impact"]["people_needing_evacuation"] >= 50,
        "injuries": facts["impact"]["injured_people"] > 0,
    }
```

```
def similarity(new_problem, old_problem):
    """Simple matching score: proportion of matching Boolean features."""
    matches = sum(new_problem[key] == old_problem[key] for key in new_problem)
    return matches / len(new_problem)


def retrieve_most_similar_case(new_problem, cases):
    best_case = None
    best_score = -1
    for case in cases:
        score = similarity(new_problem, case["problem"])
        if score > best_score:
            best_case = case
            best_score = score
    return best_case, best_score


def run_cbr(facts, cases):
    # RETRIEVE: find the case with the highest feature match.
    new_problem = get_problem_features(facts)
    matched_case, score = retrieve_most_similar_case(new_problem, cases)

    # REUSE: begin with the solution from the most similar past case.
    reused_solution = matched_case["solution"]

    # REVISE: a simple example of adapting the past solution to current facts.
    revised_solution = reused_solution
    if facts["resources"]["rescue_vehicles_available"] < 2:
            revised_solution += " Prioritize the highest-risk locations because rescue
vehicles are limited."
    if not facts["resources"]["shelter_available"]:
        revised_solution += " Identify a safe temporary evacuation site."

    # RETAIN: store the current problem and revised solution as a new case.
    new_case = {
        "name": "New Case - Current Flood Incident",
        "problem": new_problem,
        "solution": revised_solution,
    }
```

```
    cases.append(new_case)
    return matched_case["name"], score, reused_solution, revised_solution, len(cases)


def main():
    print("=" * 66)
    print("TA4: DISASTER RESPONSE DECISION SUPPORT SYSTEM")
    print("=" * 66)

    print("\nPART 1 - KNOWLEDGE BASE")
    print("Categories: hazard, impact, resources")
    for category, facts in incident_facts.items():
        print(f"  {category.title()}: {facts}")

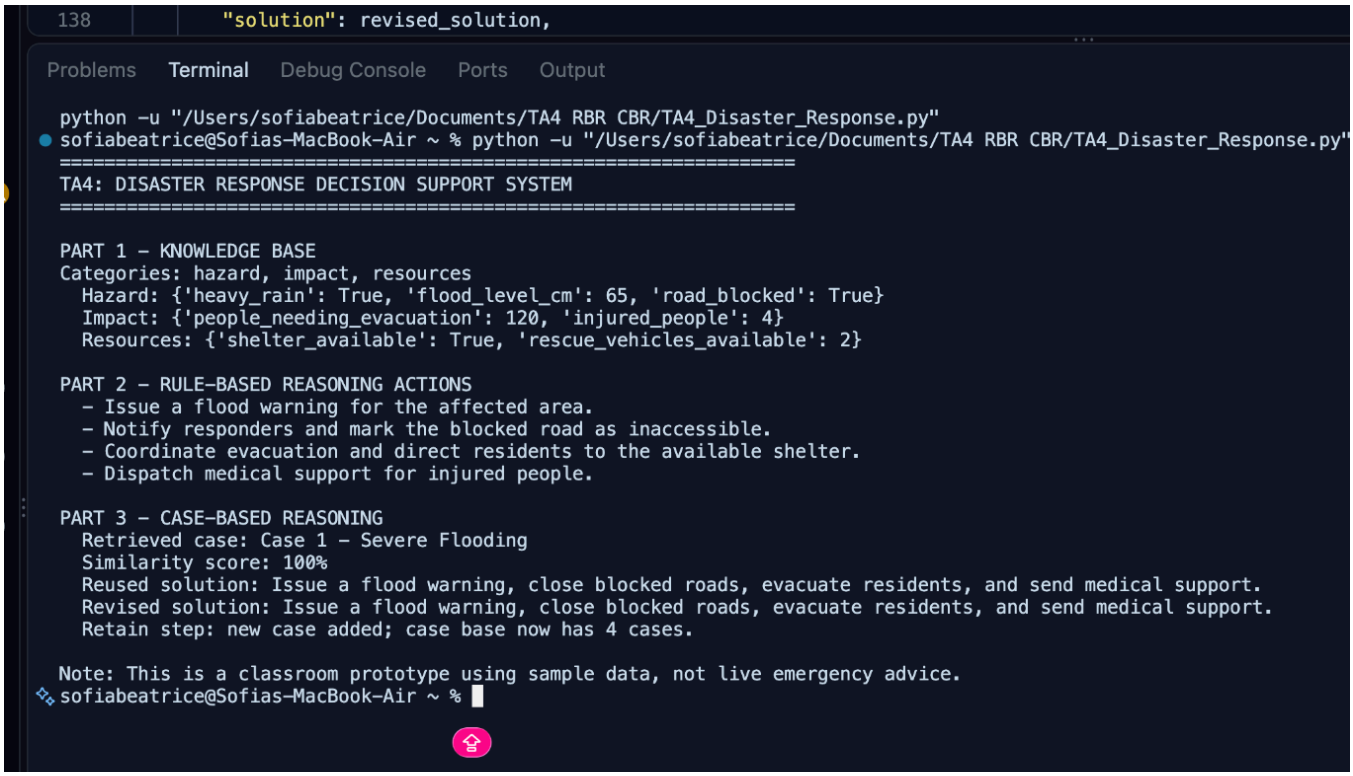
    print("\nPART 2 - RULE-BASED REASONING ACTIONS")
    actions = run_rbr(incident_facts)
    for action in actions:
        print(f"  - {action}")

    print("\nPART 3 - CASE-BASED REASONING")
    matched_name, score, reused, revised, count = run_cbr(incident_facts, case_base)
    print(f"  Retrieved case: {matched_name}")
    print(f"  Similarity score: {score:.0%}")
    print(f"  Reused solution: {reused}")
    print(f"  Revised solution: {revised}")
    print(f"  Retain step: new case added; case base now has {count} cases.")

    print("\nNote: This is a classroom prototype using sample data, not live emergency
advice.")


if __name__ == "__main__":
    main()
```

OUTPUT:


138 "solution": revised_solution,
Problems Terminal Debug Console Ports Output
python -u "/Users/sofiabeatrice/Documents/TA4 RBR CBR/TA4_Disaster_Response.py"
sofiabeatrice@Sofias-MacBook-Air ~ % python -u "/Users/sofiabeatrice/Documents/TA4 RBR CBR/TA4_Disaster_Response.py"
TA4: DISASTER RESPONSE DECISION SUPPORT SYSTEM
PART 1 - KNOWLEDGE BASE
Categories: hazard, impact, resources
Hazard: {'heavy_rain': True, 'flood_level_cm': 65, 'road_blocked': True}
Impact: {'people_needing_evacuation': 120, 'injured_people': 4}
Resources: {'shelter_available': True, 'rescue_vehicles_available': 2}
PART 2 - RULE-BASED REASONING ACTIONS
- Issue a flood warning for the affected area.
- Notify responders and mark the blocked road as inaccessible.
- Coordinate evacuation and direct residents to the available shelter.
- Dispatch medical support for injured people.
PART 3 - CASE-BASED REASONING
Retrieved case: Case 1 - Severe Flooding
Similarity score: 100%
Reused solution: Issue a flood warning, close blocked roads, evacuate residents, and send medical support.
Revised solution: Issue a flood warning, close blocked roads, evacuate residents, and send medical support.
Retain step: new case added; case base now has 4 cases.
Note: This is a classroom prototype using sample data, not live emergency advice.
sofiabeatrice@Sofias-MacBook-Air ~ %